



**UNIVERSIDADE FEDERAL DO PIAUÍ
CENTRO DE CIÊNCIAS DA NATUREZA
DISCIPLINA ARQUITETURA DE COMPUTADORES
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
PROFESSOR IVAN SARAIVA SILVA**

**ANTONIO INÁCIO DA SILVA NETO
JARDENIA MILLENA DE SOUSA ROSA**

PROGRAMAÇÃO IGUANA

TERESINA – PI

2024

CÓDIGO MAC-1

```

1  .data
2      array:      .word -1, -2, -3, -4, -5, 6, 7, 8, 9, 10
3      index:      .word 0
4      one:        .word 1
5      two:        .word 2
6      control:    .word 9
7  .text
8  MAIN:           # Início
9      LOCO array   # coloca o endereço do vetor em AC
10     SWAP         # coloca o endereço do vetor em SP e AC recebe lixo
11  LOOP:          # Loop para verificar valores atuais positivos ou negativos
12     LODL 0       # AC recebe o valor atual do vetor
13     JPOS VERIFYPOS # verificaremos se o valor atual é positivo: se sim, verificar se é par ou impar
14     JNEG VERIFYNEG # verificaremos se o valor atual é negativo: se sim, verificar se é par ou impar
15  VERIFYPOS:     # Verificar se o positivo é par ou impar
16     SUBD two     # AC recebe AC - [two]:2
17     JZER ISEVEN  # se AC agora for zero, é par.
18     JNEG ISODD   # se AC agora for negativo, é impar.
19     JUMP VERIFYPOS # se não pulou antes, é pq nem é zero e nem é negativo, continua
20  VERIFYNEG:     # Verificar se o negativo é par ou impar
21     ADDD two     # AC recebe AC - [two]:2
22     JZER ISEVEN  # se AC agora for zero, é par.
23     JPOS ISODD   # se AC agora for positivo, é impar e negativo.
24     JUMP VERIFYNEG # se não pulou antes, é pq nem é zero e nem é negativo, continua
25  ISEVEN:        # Se for par
26     LODL 0       # AC recebe o valor atual do vetor
27     #PRINTLNAC
28     INSP 1       # SP avança no vetor
29     JUMP INDEXINC # Pula para index increase
30  ISODD:         # Se for impar
31     LODL 0       # AC recebe o valor atual do vetor
32     ADDL 0       # AC recebe AC + o valor atual do vetor
33     STOL 0       # O valor atual do vetor recebe 2x o valor atual do vetor
34     #PRINTLNAC
35     INSP 1       # SP avança no vetor
36     JUMP INDEXINC # Pula para index increase
37  INDEXINC:      # Incrementar o index
38     LODD index   # AC recebe [index]
39     ADDD one     # AC recebe AC + 1
40     STOD index   # index é incrementado
41     JUMP VERIFYEND # verifica se é o fim do vetor
42  VERIFYEND:     # Verifica se tá no fim do vetor
43     LODD control # AC recebe [control]:9
44     SUBD index   # AC recebe AC - [index]
45     JPOS LOOP    # Se for positivo, verifica o próximo
46     JNEG END     # Se for negativo, chegou ao fim do vetor, finaliza
47  END:           # Encerra o programa
48     JUMP END     #HALT

```

CÓDIGO IGUANA

```

1  .data
2      array:      .word -1, -2, -3, -4, -5, 6, 7, 8, 9, 10
3      index:      .word 0
4      one:        .word 1
5      two:        .word 2
6      control:    .word 9
7      space:      .ascii " "
8  .text
9  MAIN:          # Início
10     LOCO array   # coloca o endereço do vetor em AC
11     SWAP         # coloca o endereço do vetor em SP e AC recebe lixo
12     LOOP:        # Loop para verificar valores atuais positivos ou negativos
13     LODL 0       # AC recebe o valor atual do vetor
14     JPOS VERIFYPOS # verificaremos se o valor atual é positivo: se sim, verificar se é par ou ímpar
15     JNEG VERIFYNEG # verificaremos se o valor atual é negativo: se sim, verificar se é par ou ímpar
16     VERIFYPOS:   # Verificar se o positivo é par ou ímpar
17     SUBD two     # AC recebe AC - [two]:2
18     JZER ISEVEN  # se AC agora for zero, é par.
19     JNEG ISODD   # se AC agora for negativo, é ímpar.
20     JUMP VERIFYPOS # se não pulou antes, é pq nem é zero e nem é negativo, continua
21     VERIFYNEG:   # Verificar se o negativo é par ou ímpar
22     ADDD two     # AC recebe AC - [two]:2
23     JZER ISEVEN  # se AC agora for zero, é par.
24     JPOS ISODD   # se AC agora for positivo, é ímpar e negativo.
25     JUMP VERIFYNEG # se não pulou antes, é pq nem é zero e nem é negativo, continua
26     ISEVEN:      # Se for par
27     LODL 0       # AC recebe o valor atual do vetor
28     PRINTAC
29     LODD space
30     PRINTACCHAR
31     INSP 1       # SP avança no vetor
32     JUMP INDEXINC # Pula para index increase
33     ISODD:       # Se for ímpar
34     LODL 0       # AC recebe o valor atual do vetor
35     MULD two     # AC recebe AC * [two]: 2
36     STOL 0       # O valor atual do vetor recebe 2x o valor atual do vetor
37     PRINTAC
38     LODD space
39     PRINTACCHAR
40     INSP 1       # SP avança no vetor
41     JUMP INDEXINC # Pula para index increase
42     INDEXINC:    # Incrementar o index
43     LODD index   # AC recebe [index]
44     ADDD one     # AC recebe AC + 1
45     STOD index   # index é incrementado
46     JUMP VERIFYEND # verifica se é o fim do vetor
47     VERIFYEND:   # Verifica se tá no fim do vetor
48     LODD control # AC recebe [control]:9
49     SUBD index   # AC recebe AC - [index]
50     JPOS LOOP    # Se for positivo, verifica o próximo
51     JNEG END     # Se for negativo, chegou ao fim do vetor, finaliza
52     END:         # Encerra o programa
53     HALT

```